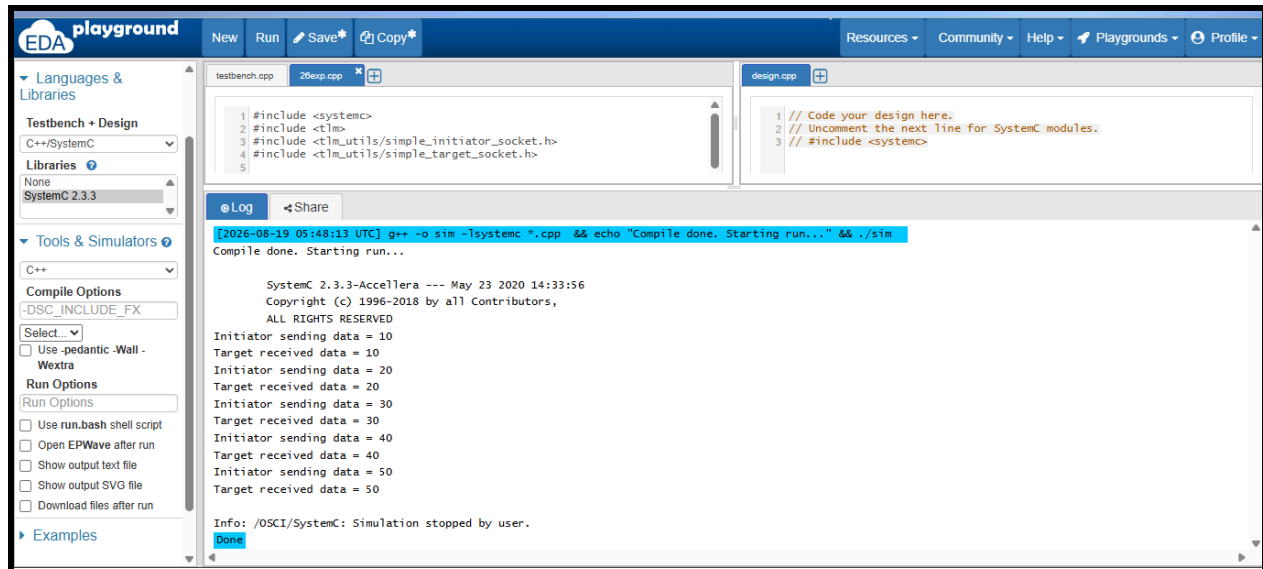




C++ Program:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>
```

```
using namespace sc_core;
using namespace tlm;
```

```
// TARGET
struct target : sc_module
{
    tlm_utils::simple_target_socket<target> socket;

    SC_CTOR(target)
    {
        socket.register_b_transport(
            this,
            &target::b_transport
        );
    }

    void b_transport(
        tlm_generic_payload& trans,
        sc_time& delay)
    {
        int* data =
```

```

        reinterpret_cast<int*>(trans.get_data_ptr());

std::cout << "Target received data = "
        << *data << std::endl;

delay += sc_time(10, SC_NS);

trans.set_response_status(TLM_OK_RESPONSE);
    }
};

```

```

// INITIATOR
struct initiator : sc_module
{
    tlm_utils::simple_initiator_socket<initiator> socket;

    SC_CTOR(initiator)
    {
        SC_THREAD(process);
    }

    void process()
    {
        for (int i = 1; i <= 5; i++)
        {
            int data = i * 10;

            tlm_generic_payload trans;
            sc_time delay = SC_ZERO_TIME;

            trans.set_command(TLM_WRITE_COMMAND);

            trans.set_address(0);

            trans.set_data_ptr(
                reinterpret_cast<unsigned char*>(&data)
            );

            trans.set_data_length(sizeof(data));

            std::cout << "Initiator sending data = "
                << data << std::endl;

```

```

        socket->b_transport(trans, delay);

        wait(delay);
    }

    sc_stop();
}
};

```

```

// TOP MODULE
SC_MODULE(top)
{
    initiator init;
    target targ;

    SC_CTOR(top)
        : init("initiator"),
          targ("target")
    {
        init.socket.bind(targ.socket);
    }
};

```

```

// MAIN
int sc_main(int argc, char* argv[])
{
    top t("top");

    sc_start();

    return 0;
}

```